



Presenter Guide — Session 2: Make It Interactive

Duration: 90 minutes · **Outcome:** every student builds a working in-memory To-Do app (add / complete / delete / count).

At a glance

Block	Time	Topic
1	0:00–0:07	Welcome back · recap · today's goal (show the finished app)
2	0:07–0:14	State & <code>useState</code> · counter live-demo
3	0:14–0:18	Rules of state · responding to taps (<code>onPress</code>)
4	0:18–0:23	<code>TextInput</code> (controlled) · <code>addTask</code>
5	0:23–0:31	Arrays in state: <code>spread / filter / map</code> · <code>FlatList</code>
6	0:31–0:38	Components & <code>props</code> (<code>TodoItem</code>) · derived values · live-code
7	0:38–1:18	Activity 2: build the To-Do (and circulate)
8	1:18–1:30	The "data is gone!" cliffhanger · recap · take-home



Compressible: the number-example slides (filter/map with `[1,2,3]`) can be shortened if the room is following. **Never skip** the immutability point (copy, don't mutate) — it's the root of most bugs.

Before class

- ☐ `git checkout session-2 && npm install && npx expo start` runs the finished To-Do on your phone.
- ☐ You can show **both** the counter demo and the full app.
- ☐ Re-confirm everyone's setup still works from last week.

Block-by-block

Block 1 — Recap & goal (0:00–0:07)

- 1-line recap; invite a volunteer to show their take-home card.
- Pivot: "last week's screen was **frozen**." Then **show the finished To-Do** on your phone — the destination.

Block 2 — State & useState (0:07–0:14)

 **ELABORATE:** State = "what's true right now," the app's live memory. **Change the data** → **the screen follows**. Decode `const [count, setCount] = useState(0)` slowly: it returns the value + a setter; the argument is the start value.

- **Live-code the counter.** Tap on your phone, watch it change.
- **Break it on purpose:** `count = count + 1` → nothing happens. That failure motivates the rules.

Block 3 — Rules of state + taps (0:14–0:18)

 **ELABORATE:** The **ONLY** way to change state is the setter. Direct assignment is silently ignored. Changing state re-runs the component (a "re-render").

- `<Button>` (basic) vs `<TouchableOpacity>` (stylable). We use TouchableOpacity.
- **The () trap:** `onPress={addTask}`  vs `onPress={addTask()}`  (runs immediately). Show the bug.

Block 4 — TextInput (0:18–0:23)

 **ELABORATE:** A controlled input is a mirror of state: `value={text}` shows it, `onChangeText={setText}` pushes keystrokes back. Same state⇌UI loop as the counter.

- Walk `addTask` line by line: trim/guard empty → build a task **object** → add to array → clear input.

Block 5 — Arrays in state + FlatList (0:23–0:31)

 **ELABORATE — most important block:** three array tools = in-memory CRUD.

- **Add:** `setTasks([newTask, ...tasks])` (spread copies the old ones)
 - **Remove:** `setTasks(tasks.filter(t => t.id !== id))`
 - **Update/toggle:** `setTasks(tasks.map(t => t.id === id ? {...t, done: !t.done} : t))` Why copy? React repaints only when handed a **new** array via the setter. Mutating the old one = no repaint.
- Ground `filter` (keep some) and `map` (change each) with plain number examples first.
 - `FlatList`: `data` + `renderItem` + `keyExtractor`. It only renders what's visible (fast). Every row needs a unique key.

Block 6 — Components, props, derived values (0:31–0:38)

 **ELABORATE:** `props` are a component's inputs (like function arguments). `App` owns the data + functions; it passes each `TodoItem` a `task` and `onToggle` / `onDelete`. **Data flows down; events call back up.** Derived values: `remaining` is computed from `tasks` each render — **don't** store it as separate state (it could drift out of sync). *Derive, don't duplicate.*

- **Live-code the full app** (~8 min), reusing every piece just taught.
-

✓ Full solution code (matches **session-2** branch **App.js**)

```
import { StatusBar } from 'expo-status-bar';
import { useState } from 'react';
import {
  FlatList, StyleSheet, Text, TextInput, TouchableOpacity, View,
} from 'react-native';

function TodoItem({ task, onToggle, onDelete }) {
  return (
    <View style={styles.item}>
      <TouchableOpacity style={styles.itemLeft} onPress={onToggle}>
        <Text style={styles.checkbox}>{task.done ? '✓' : '☐'}</Text>
        <Text style={styles.itemText}>{task.title}</Text>
      </TouchableOpacity>
      <TouchableOpacity onPress={onDelete} hitSlop={10}>
        <Text style={styles.delete}>🗑️</Text>
      </TouchableOpacity>
    </View>
  );
}

export default function App() {
  const [text, setText] = useState('');
  const [tasks, setTasks] = useState([
    { id: 1, title: 'Welcome to your To-Do app 🙌', done: false },
    { id: 2, title: 'Tap a task to mark it done', done: false },
  ]);

  function addTask() {
    const title = text.trim();
    if (title === '') return; // ignore empty input
    const newTask = { id: Date.now(), title, done: false };
    setTasks([newTask, ...tasks]); // new array, new task on top
    setText(''); // clear the box
  }

  function toggleTask(id) {
    setTasks(tasks.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
  }

  function deleteTask(id) {
    setTasks(tasks.filter((t) => t.id !== id));
  }

  const remaining = tasks.filter((t) => !t.done).length;

  return (
```

```

<View style={styles.screen}>
  <StatusBar style="light" />

  <View style={styles.header}>
    <Text style={styles.title}>My Tasks</Text>
    <Text style={styles.subtitle}>{remaining} left to do</Text>
  </View>

  <View style={styles.inputRow}>
    <TextInput
      style={styles.input}
      placeholder="Add a task..."
      placeholderTextColor="#94A3B8"
      value={text}
      onChangeText={setText}
      onSubmitEditing={addTask}
      returnKeyType="done"
    />
    <TouchableOpacity style={styles.addBtn} onPress={addTask}>
      <Text style={styles.addBtnText}>+</Text>
    </TouchableOpacity>
  </View>

  <FlatList
    data={tasks}
    keyExtractor={({item}) => String(item.id)}
    contentContainerStyle={styles.list}
    ListEmptyComponent={
      <Text style={styles.empty}>No tasks yet. Add one above! ✨</Text>
    }
    renderItem={({ item }) => (
      <TodoItem
        task={item}
        onToggle={() => toggleTask(item.id)}
        onDelete={() => deleteTask(item.id)}
      />
    )}
  />
</View>
);
}

const styles = StyleSheet.create({
  screen: { flex: 1, backgroundColor: '#F1F5F9', paddingTop: 60 },
  header: { paddingHorizontal: 20, paddingBottom: 12 },
  title: { fontSize: 32, fontWeight: 'bold', color: '#0F172A' },
  subtitle: { fontSize: 15, color: '#64748B', marginTop: 2 },
  inputRow: { flexDirection: 'row', paddingHorizontal: 20, paddingBottom: 12, gap: 10 },
  input: {
    flex: 1, backgroundColor: 'FFFFFF', borderRadius: 12, paddingHorizontal: 16,

```

```
paddingVertical: 12, fontSize: 16, color: '#0F172A', borderWidth: 1, borderColor: '#E
},
addBtn: { width: 48, backgroundColor: '#4338CA', borderRadius: 12, alignItems: 'center'
addBtnText: { color: '#FFFFFF', fontSize: 28, fontWeight: '600', lineHeight: 30 },
list: { paddingHorizontal: 20, paddingBottom: 40 },
item: { flexDirection: 'row', alignItems: 'center', backgroundColor: '#FFFFFF', borderR
itemLeft: { flexDirection: 'row', alignItems: 'center', flex: 1, gap: 12 },
checkbox: { fontSize: 20 },
itemText: { fontSize: 16, color: '#0F172A', flexShrink: 1 },
itemTextDone: { textDecorationLine: 'line-through', color: '#94A3B8' }, // ← used in th
delete: { fontSize: 20, paddingLeft: 12 },
empty: { textAlign: 'center', color: '#94A3B8', marginTop: 40, fontSize: 16 },
});
```

Block 7 — Activity 2 facilitation (0:38–1:18)

Common student errors & fixes

Symptom	Cause	Fix
Button fires on load / loops	<code>onPress={addTask()}</code>	Remove the <code>()</code> → <code>onPress={addTask}</code>
Typing does nothing / can't edit	Forgot <code>value</code> or <code>onChangeText</code>	Wire both: <code>value={text}</code> <code>onChangeText={setText}</code>
List never updates after add	Mutated array (<code>tasks.push</code>)	Use <code>setTasks([newTask, ...tasks])</code>
Warning: "unique key prop"	Missing/duplicate keys	<code>keyExtractor={(item) => String(item.id)}</code>
All rows toggle/delete together	Compared wrong id, or no <code>id</code>	Each task needs a unique <code>id</code> (<code>Date.now()</code>)
App resets on save	That's hot reload — expected	Real persistence comes in Session 3
Keyboard covers the input	—	Fine for now; mention <code>KeyboardAvoidingView</code> exists

 **Facilitation:** Most bugs today are the `()` trap and array mutation. When a student is stuck, ask "are you making a **new** array?" and "did you pass the function or **call** it?"

Block 8 — Cliffhanger & wrap (1:18–1:30)

- **Everyone fully closes and reopens the app.** Tasks reset to the samples. Let it land.
- "State lives in memory; memory clears. Next week the app gets **real memory** — a database."
- **Take-home:** add a **"Clear done"** button + apply the existing `itemTextDone` strike-through style.
- Active recall: "How do we add to an array in state?" · "What makes a row tappable?"

Answer key — take-home (stretch additions)

```
// 1) Strike-through completed tasks – wire up the existing style:
<Text style={[styles.itemText, task.done && styles.itemTextDone]}>
  {task.title}
</Text>

// 2) "Clear done" button (place near the header):
function clearDone() {
  setTasks(tasks.filter((t) => !t.done)); // keep only the not-done ones
}

<TouchableOpacity onPress={clearDone}>
  <Text>Clear done</Text>
</TouchableOpacity>
```

Full marks for: a new array via the setter (no mutation), correct conditional style, and the button working.